Proc. of the 4th International Conference on Electrical, Communication and Computer Engineering (ICECCE)
30-31 December 2023, Dubai, UAE

2023 International Conference on Electrical, Communication and Computer Engineering (ICECCE) | 979-8-3503-6969-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICECCE61019.2023.10442325

Design and Analysis of a Hardware Accelerator with FPU-Based Runge-Kutta Solvers

Soham Bhattacharya
Department of Electrical and Computer Engineering
Rowan University
New Jersey, USA
bhatta22@students.rowan.edu

Dwaipayan Chakraborty
Department of Electrical and Computer Engineering
Rowan University
New Jersey, USA
chakraborty@rowan.edu

***Abstract*—In several high-speed computer applications, ordinary differential equations (ODEs) are often utilized. Modern CPUs, however, usually offer a modest throughput for these systems. This paper discusses the implemented design and analysis of a hardware accelerator including second-order, third-order, and fourth-order Runge-Kutta ODE solvers for improving the computing speed and accuracy. The implementation of the hardware accelerators has been done using the VHDL programming language and has been deployed on the Zynq ZC702 FPGA evaluation board. A comprehensive comparative analysis of different Runge-Kutta hardware accelerators, highlighting factors including timing performance, hardware resource usage, and the total on-chip power consumption has been given. According to the experimental findings, the fourth-order Runge-Kutta equation has a significant power consumption, rising by an estimated 51.37 percent and 12.445 percent when comparing second-order and third-order Runge-Kutta solvers at a clock rate of 100 MHz, respectively. This study contributes to the enhancement of ODE solutions in high-performance computing applications.**

Keywords—Hardware Accelerator, Runge Kutta Equations, ODE, RISC-V.

I. INTRODUCTION

Ordinary differential equations, or ODEs, are a crucial tool for modeling and simulating dynamic systems in many different scientific and technical fields. A precise foundation for describing natural processes can be provided by these equations. Although certain ODEs may be solved analytically, this approach is frequently unfeasible due to the complexity of real-world systems. Numerical techniques save precious time in these situations by enabling us to efficiently analyze system behavior and approximate solutions. With every ongoing generation of microprocessors, there has been a widespread expectation of notable speed gains in the current era of high-performance computing. Software optimization for these architectures has become more important due to emerging trends like multi-core design and the usage of accelerators like Field-Programmable Gate Arrays (FPGAs) and General-Purpose Graphics Processing Units (GPGPUs). The deceleration of Dennard scaling and Moore's law [1, 2] provide difficulties to the historical progress made possible by superscalar computer architectures [3] and Moore's law [4]. Application-specific co-processors are getting more and more important as computer architecture gets more varied [5]. Hardware accelerators, frequently built into CPUs, provide significant performance improvements and are tailored for certain workloads. When compared to intricate CPUs, these accelerators improve program performance and energy efficiency. Each hardware accelerator performs a single task or a limited set of related tasks. Hardware accelerators are intended for a specific use and offer a substantial performance benefit. Sometimes the hardware is incorporated inside the CPU, and also newer CPUs have dedicated internal parts that speed particular sorts of arithmetic required for activities such as video encoding and encryption. Hardware accelerators are specially designed to improve performance. Our program will perform considerably more smoothly or quickly than it would otherwise. A compact and specific chip will always consume less power than a huge and complicated CPU.

The goal of this study is to construct hardware accelerators for solving the Runge-Kutta family of non-linear Ordinary Differential equations, which is widely utilized in applications linked to numerous engineering and scientific challenges. The accelerators have been created with the help of a Hardware Descriptive Language known as "Very high-speed integrated circuit hardware description language" (VHDL), which incorporates RISC-V CPUs and all the parts associated with them, shown in Fig. 4. The accelerator is typically deployed on the Zynq ZC702 FPGA Evaluation Kit. The paper is outlined as follows. The context of the literature review on how to build and create the hardware

979-8-3503-6969-4/23/$31.00©2023 IEEE

Authorized licensed use limited to: The University of Toronto. Downloaded on April 20,2026 at 06:36:20 UTC from IEEE Xplore. Restrictions apply.

accelerator for various types of scientific computing approaches is covered in Section II. Section III describes an overview of the Runge Kutta family of equations for solving ordinary differential equations. In Section IV, the architectural description of the hardware accelerator for the RK family of equations has been discussed, followed by the power and performance metrics analysis in Section V. Section VI concludes by summarizing the findings.

II. RELATED WORK

In [6], Jack et al. have presented some new methodologies to use these new architectures, especially some special purpose (eg. General Purpose Units (GPUs)) and reconfigurable computing (such as FPGAs) along with multi-core architectures in CFD for Air Force Applications. In the recent technology world, the core doubling has come into play whereas in the past, clock doubling was there for every two years. Multi-core is the new solution to the problems related to performance of the device. It is now known that the widespread improvement of the central processing unit (CPU) - based devices has come from the multi-core platforms. For example, IBM has gained the path to introduce their own heterogeneous multi-core architecture, the CELL Broadband Engine. Nowadays, the floating point performance on FPGA and GPU is more relevant than that of CPU. Non floating point problems are generally solved by the FPGA where the massive reconfigurable fabric of the FPGA is useful for integers, fixed point, boolean or custom representations. GPUs solve the SIMD (single instruction, multiple data) problems. CPU multi-cores are the excellent general purpose platforms that have the most advanced programming model, which is user-friendly to the developers till date. A. Hollabough et al. [7] have discussed the design of the hardware accelerator for solving the Lotka-Volterra family of ordinary differential equations (ODEs) using Chisel language and Chipyard environment. Hardware accelerators were used to reduce the energy consumption and for increasing the performance for a specific type of computation. In this paper, the proposed accelerator includes a co-processor interfaced with a General purpose processor. The co-processor will implement the numerical method for the Lotka-Volterra differential equation. The co-processor will include a control unit which has a state machine. The paper also describes the comparison between Lotka-Volterra using different numerical methods like Euler method, RK2 and RK4 methods, function of the accelerator and its configuration. Also, power dissipation, performance and logic utilization are also shown. In the paper [8], a promising system with an open curve of equilibrium points has been presented for chaotic secure communications. By computing the spectrum and bifurcation diagram of the Lyapunov exponents, the authors demonstrated the dynamics of this novel chaotic oscillator, which was constructed using an FPGA. Power series decomposition was used to approach the exponential function, and adders and multipliers are used to implement it inside the FPGA. Also, a chaotic system can be implemented using discrete or integrated circuits or FPGAs, stated in [9,10]. Asana et al. [11] have published a framework for modeling biological cellular processes described by ODEs on FPGAs. in their 2013 magazine. Due to the limited application space and the resulting limited choice of relevant ODEs, the authors were able to manually develop a solution for each RTL function. The proposed framework combines static elements and handwritten solutions to create the final FPGA design. Fasih et al. have suggested approaches for solving ODE on FPGA in [12]. By resolving Rossler equations, the authors have provided a mechanism for resolving higher order differential equations. Furthermore, the authors had chosen a fixed point representation strategy rather than a floating point one for all internal signals in order to achieve the needed precision. The Verilog Hardware Description language has been used to manually describe the codes [11] and [12].

III. RUNGE KUTTA ODE SOLVERS

Runge Kutta numerical methods are widely used to solve ordinary differential equations (ODEs). Among the Runge Kutta methods, RK2. RK3. RK4 are commonly used variants for approximating the solutions to the differential equations. RK2 or second order Runge Kutta numerical method [13] is a simple but effective method in which at each step, the solution is updated by approximating the derivative at a particular location in the ODE using a weighted average of two slopes. This approach is well-liked for several real-world applications because of its reasonable accuracy and computing efficiency. In comparison to RK2, the third-order Runge-Kutta technique, or RK3, [14] provides better accuracy. For an update to the answer, three slopes and their weighted averages need to be calculated. As a result of the increased analyzing effort, the derivative is approximated more accurately, which makes it appropriate for ODEs with steeper gradients or when greater accuracy is needed. Runge Kutta fourth order differential equation solver method or RK4 method is used to approximate the value of ‘y’ with a given value of ‘x’. RK4 methods are generally used to solve the initial-value problems of differential equations.

Authorized licensed use limited to: The University of Toronto. Downloaded on April 20,2026 at 06:36:20 UTC from IEEE Xplore. Restrictions apply.

The general form to represent RK4 numerical method is :

$$K_1 = f(x_n, y_n) \quad (1)$$

$$K_2 = f(x_n + h/2, y_n + hK_1/2) \quad (2)$$

$$K_3 = f(x_n + h/2, y_n + hK_2/2) \quad (3)$$

$$K_4 = f(x_n + h, y_n + hK_3) \quad (4)$$

and,

$$y_{n+1} = y_n + (h/6)(K_1 + 2K_2 + 2K_3 + K_4) + O(h^5) \quad (5)$$

$$x_{n+1} = x_n + h \quad (6)$$

where, the value of 'n' varies from 0 to $(x - x_0)/h$ and 'h' is the step size. The RK4 approach uses the current value (y_n) and four intermediary slopes to approximate the next value y_{n+1}. K_1 is determined by the slope at the beginning of the interval, K_2 by halfway through, K_3 in the center, and K_4 at the end of the interval. RK4 is a reliable approach that provides precision for solving numerical differential equations when combined with a dynamic step-size technique. RK2 is simple to use and effective, however RK3 offers higher accuracy.

IV. ARCHITECTURE DESCRIPTION

A. Overview

Three accelerators have been developed for the second or third or fourth order Runge Kutta solvers. Each accelerator module consists of a second or third or fourth order Runge Kutta Module Unit, a memory module unit, a register file, and a control unit, shown in Fig. 4. The accelerator is designed using Xilinx Vivado software suite by AMD, Standard Edition, which has been used to develop the accelerator and to determine the metrics for the analysis of the accelerator. The accelerator has been designed for a simple ordinary differential equation solved using RK4 numerical method, stated in (7).

$$f = x^2 - y^2 \quad (7)$$

Each accelerator is made up of several modules, including the functional computation block (FCB), where the function of the ODE is indicated, the K-Blocks, where the ordinary differential equations of the coefficients utilizing the FCB have been executed, and the Runge Kutta Modules, where the blocks compute the values of x_{n+1} and y_{n+1}. Additionally, a memory unit that holds the value of the approximation of x_{n+1} and y_{n+1} has been introduced. A control unit has been developed to implement the custom instructions in accordance with the RISC-V ISA standard. To initialize the values of the particular units to the specific registers, the register file has been implemented. The hardware accelerators take some initial set of values of x_{in} and y_{in} as 1 and 2 respectively. A third party vendor AXI4 stream single precision floating point IP support from Xilinx Vivado has been used to generate the results for all the respective values of x_{n+1}, y_{n+1}, h, and the four coefficients K_1, K_2, K_3, K_4. The accelerators will run for 'n' number of iterations after giving single custom instructions to initialize, flush, update and store for each case according to the RISC-V ISA format.

B. Hardware Implementation of the Accelerator

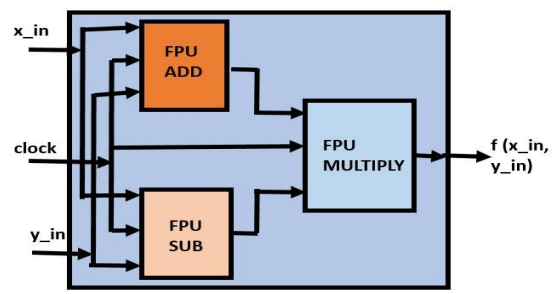

Fig. 1. Function Computation Block (FCB).

In this paper, the implementation of the fourth order Runge Kutta solver has been described. The Function Computation Block (FCB) has been initially developed. The function described in (7) has been computed using the FCB in Fig. 1 to obtain the value of f. The FCB has been implemented using three single precision floating point units: FPU_ADD, FPU_SUB, and FPU_MULTIPLY. Each of the floating point units such as FPU_ADD, FPU_SUB,

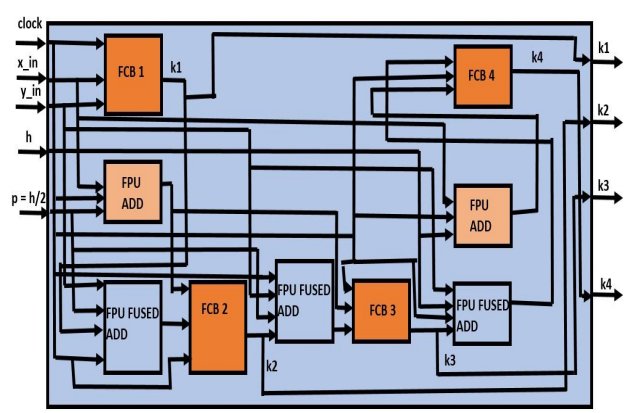

Fig. 2. K-Block.

Authorized licensed use limited to: The University of Toronto. Downloaded on April 20,2026 at 06:36:20 UTC from IEEE Xplore. Restrictions apply.

FPU_MUL has a latency of five clock cycles for each hardware accelerator design.

The K-Block consists of all the operations associated with the four coefficients: K1, K2, K3, K4, stated in (1), (2), (3), (4) shown in Fig. 2. In this K-Block, FCB has been used along with the FPU_ADD modules and sets of Multiply-Accumulate (MAC) units, termed as 'FPU_FUSED_ADD' blocks. A latency of ten clock cycles has been used for the MAC units for each accelerator design. The FCB blocks compute the function f and then utilize it in generating the value of K_1, K_2, K_3, K_4. Hardware MAC units, which were made up of an adder and a multiplier, handle MAC operations. After multiplying the first two inputs, the third input has been added to the result. The Vivado environment's FPU_Fused floating point IP support has been used to produce the 32-bit floating point MAC operations. To create the final value of x_{n+1} and y_{n+1}; the RK4 module in Fig. 3 has been designed utilizing the K-block, together with FPU_ADD, FPU_MUL, and MAC units. Here, the terms x_{out} and y_{out}, respectively, demonstrate how the values of x_{n+1} and y_{n+1} are approximated. A comparative analysis of the number of Xilinx Vivado single precision floating point IP support used to build the hardware accelerator for solving (7) using second order, third order and fourth order Runge Kutta numerical methods have been represented in Table I.

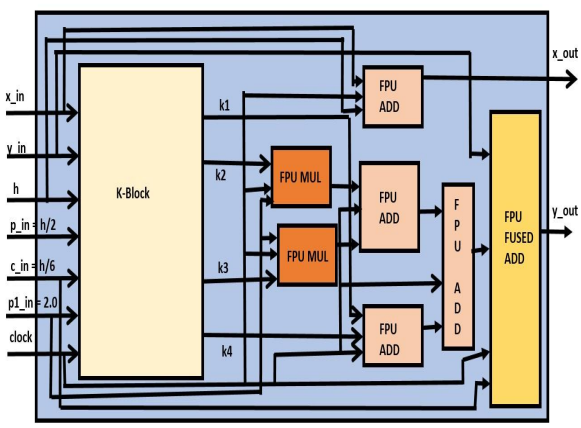

Fig. 3. Design of Runge Kutta fourth order equation (RK4) module.

TABLE I.
Representation of Xilinx Vivado's floating point IP support used for implementing the hardware accelerator designs using RK2, RK3 and RK4 methods.

ODE Solvers	FPU_ ADD	FPU_ SUB	FPU_ MUL	MAC Units
RK2	6	1	4	X
RK3	5	2	5	4
RK4	7	1	2	4

C. *Accelerator Microarchitecture*

The hardware accelerator, shown in Fig. 4, has been initialized with reading the values of the parameters of 32-bits each of x_{in}, y_{in}, h, $h/2$, $h/6$, $p1_in$ stored at the specific locations of the source register, rs1. P_in has been used in Fig. 4 to represent h/2, while c_in has been utilized to denote h/6. P1_in has been used to represent integer 2 for doing the operation of the coefficient of K_2 and K_3.The control unit has been designed according to the RISC-V I-Type Instruction Format. Whenever the instructions have been set to check for the approximations of x_{n+1} and y_{n+1}, the control unit will receive the instructions, and the control signals flush, write_en, init will instruct the memory to store and update the approximations of x_{n+1} and y_{n+1} in the memory addresses. Four custom instructions that have been utilized for implementing the hardware accelerators for the function (7) were RKI, RKF, RKU, and RKS. In RKI, known as "RK Initialize" instruction format, a register file will generate 32-bit data as "content" of x_in, y_in, h, p_in, c_in, p1_in to the memory unit to generate the result for the RK4 module. This will be sent to the output port "contents", in Fig. 4. RKI is independent of flush signals. However, in order to flush, there must be something in memory, for which RKI needs to have been completed. In the instruction format for RKF, ("RK Flush" instruction), a bit known as a "flush" bit has been assigned. One bit control signal, when it's '1', will flush out the contents of x_in, y_in, h, p_in, c_in and p1_in to the RK4 module. All the RK solver input parameters have been stored in the memory address, ranging from 0 to 20. The "RK Flush" instruction format is what this is. When flush is set to '0', the outputs x_{out} and y_{out} from the RK4 module will be updated and saved in the mem[0] and mem[4] locations for x_{n+1} and y_{n+1}, respectively. That is called the "RK Update"(RKU) instruction format. All the results will be stored in new addresses of memory using the RK store (RKS) instruction format.

V. PERFORMANCE ANALYSIS

The hardware accelerator has been synthesized to work very efficiently with Zynq-ZC702 FPGA

Authorized licensed use limited to: The University of Toronto. Downloaded on April 20,2026 at 06:36:20 UTC from IEEE Xplore. Restrictions apply.

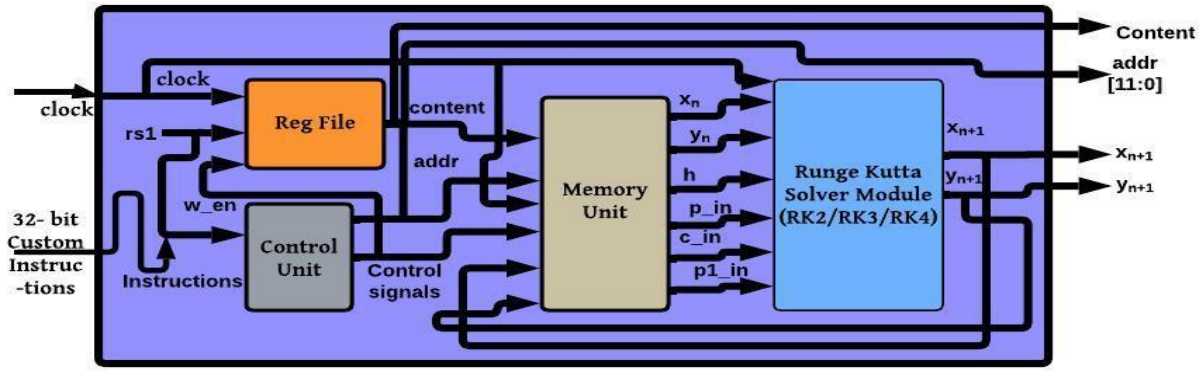

Fig. 5. A comparative graphical analysis of the total on-chip power consumption for the hardware accelerators using the Runge Kutta ODE solvers.

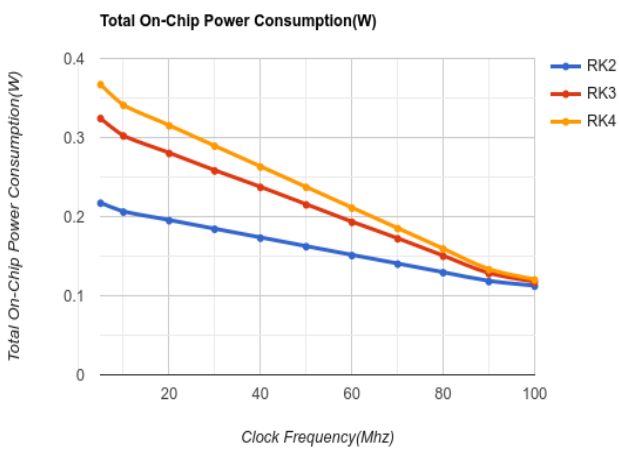

Fig. 4. The architecture design of the Hardware Accelerator of an Ordinary Differential Equation with the second-order or the third-order or the fourth-order Runge Kutta Solver.

Evaluation Board (xc7z020clg484-1) due to its increased number of I/O pins. The power, hardware utilization percentage and timing metrics have been represented in detail in the subsections.

A. Power Analysis

The Xilinx Vivado Power estimator has been used to calculate the power for the hardware accelerator created for all the RK numerical functions defined in (7). A comparative graphical analysis of the hardware accelerators of all the Runge Kutta family of ODE solvers have been given according to the clock frequency in Fig. 5. All the experiments have been measured using 0 to 100 Mhz clock frequency and the power analyses have been done based on that frequency measure. According to that, the maximum power consumption has been determined for the fourth order Runge Kutta method, which is 0.367W. Apart from it, for the third order RK hardware accelerator, the total on-chip power is 0.324W and for the second order RK hardware accelerator, the estimated power is 0.217W for 100Mhz clock frequency. After analysis the power consumption for all the three hardware accelerators design, RK4 accelerator holds the maximum dynamic source usage of 71 percent and static source usage of 29 percent than the RK2 (dynamic:51%, static:49%) and RK3 (dynamic:67%, static: 33%) accelerators. The experimental results show that the fourth-order Runge-Kutta equation consumes a substantial amount of power; it increases by an estimated 51.37 percent when using second-order solvers and 12.445 percent when using third-order solvers at a 100 MHz clock rate.

B. Hardware Resource Utilization

The look-up table (LUT) percentage used for the design of the hardware accelerators is higher for the RK4 (11%) than the other two solvers where RK2 and RK3 have 4 percent and 9 percent of LUT usages respectively. All the other hardware utilization resources of the hardware accelerator designs such as look-up tables usages for the designs distributed RAM (LUTRAM), which means the LUTs can be used as synchronous RAM, flip- flops(FF), multipliers (DSP), input outputs (IO), high fanout buffer (BUFG), have been shown using a bar chart representation in Fig. 6.

C. Timing Summary

The detailed timing analysis for the maximum and minimum setup time, hold time and pulse width analysis for the second order, third order and fourth order Runge Kutta hardware accelerator designs has been given in Table II. Fourth order Runge Kutta hardware accelerator (RK4) has the maximum setup time of 3.935 ns whereas second order Runge Kutta accelerator (RK2) has a maximum hold time of 0.554 ns. In case of minimum setup and hold time, RK2 has

Authorized licensed use limited to: The University of Toronto. Downloaded on April 20,2026 at 06:36:20 UTC from IEEE Xplore. Restrictions apply.

the minimum setup time, and hold time, which are 2.810 ns and 0.197 ns respectively.

TABLE II.
Timing analysis of all the three hardware accelerator designs.

HW Accelerators	Setup Time (Max) (ns)	Setup Time (Min) (ns)	Hold Time (Max) (ns)	Hold Time (Min) (ns)	Pulse Width (ns)
RK2	3.100	2.810	0.554	0.197	4.500
RK3	3.699	3.401	0.410	0.345	4.020
RK4	3.935	3.661	0.024	0.052	4.020

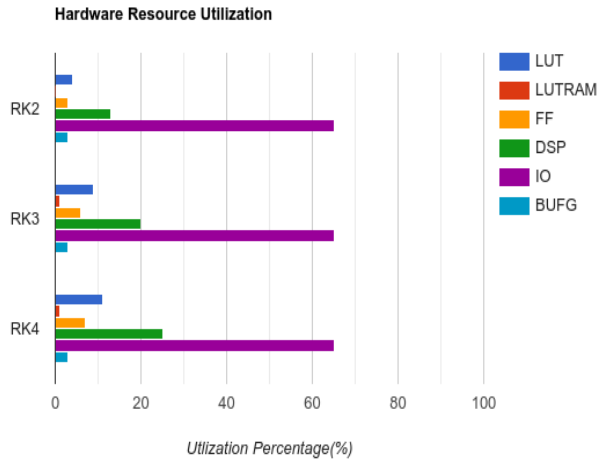

Fig. 6. Hardware resource utilization summary for all the hardware accelerator designs.

V. CONCLUSION AND FUTURE WORK

In this study, three hardware accelerators for the second, third and fourth order Runge Kutta ODE solvers have been designed and implemented. A comparative analysis of the power, hardware resource utilization and timing summary have been provided for all the hardware accelerators designed. All the work has been done using VHDL language in the Xilinx Vivado Environment and deployed on the Zynq-ZC702 FPGA Evaluation Board. It has been seen that the hardware accelerator runs more efficiently using the Xilinx Vivado single precision floating point IP support. With anticipated power usages of 51.37 percent when using second-order solvers and 12.445 percent when using third-order solvers at a 100 MHz clock rate, the Runge Kutta fourth-order hardware accelerator (RK4) holds the maximum on-chip power consumption. This study on hardware ODE solvers may be expanded to cover typical variants of the other ODE systems that are related. At the moment, this article plays a vital role in improving the findings' ability to run on other FPGA boards. It will be intriguing to follow up on pipelining the architecture's process and integrating the hardware accelerator into a system-on-a-chip (SOC). Another area of research may be integrating the work into a general-purpose RISC-V core and figuring out which one performs best at a given clock speed. By benchmarking our design of the hardware accelerator with a general-purpose RISC-V processor, there is much more tuning that can be done to get greater performance.

REFERENCES

[1] M. Bohr, “A 30 year retrospective on dennard’s mosfet scaling paper,” IEEE Solid-State Circuits Society Newsletter, vol. 12, no. 1, pp. 11–13, 2007.

[2] H. Esmaeilzadeh, E. Blem, R. S. Amant, K. Sankaralingam, and D. Burger, “Dark silicon and the end of multicore scaling,” in 2011 38th Annual international symposium on computer architecture (ISCA), IEEE, 2011, pp. 365–376.

[3] G. Blake, R. G. Dreslinski, and T. Mudge, “A survey of multicore processors,” IEEE Signal Processing Magazine, vol. 26, no. 6, pp. 26–37, 2009.

[4] R. R. Schaller, “Moore’s law: past, present and future,” IEEE spectrum, vol. 34, no. 6, pp. 52–59, 1997.

[5] J. Vetter, R. Brightwell et al., “Extreme heterogeneity 2018: DOE ASCR basic research needs workshop on extreme heterogeneity (2018).”

[6] Dongarra, J., Moore, S., Peterson, G., Tomov, S., Allred, J., Natoli, V., Richie, D. "Exploring New Architectures in Accelerating CFD for Air Force Applications," *Proceedings of the DoD HPCMP User Group Conference*, Seattle, Washington, July 14-17, 2008

[7] A. Hollabough and D. Chakraborty, "An Open-Source Co-processor for Solving Lotka-Volterra Equations," *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, Austin, TX, USA, 2022, pp. 1690-1694.

[8] Tlelo-Cuautle, E., de la Fraga, L.G., Pham, VT. et al. Dynamics, FPGA realization and application of a chaotic system with an infinite number of equilibrium points. Nonlinear Dyn 89, 1129–1139 (2017). https://doi.org/10.1007/s11071-017-3505-2

[9] Tlelo-Cuautle, E., de la Fraga, L.G., Rangel-Magdaleno, J.: Engineering Applications of FPGAs. Springer, Berlin (2016)

[10] Trejo-Guerra, R., Tlelo-Cuautle, E., Carbajal-Gómez, V.H., Rodriguez-Gomez, G.: A survey on the integrated design of chaotic oscillators. Appl. Math. Comput. 219(10), 5113– 5122 (2013)

[11] Osana Y, Fukushima T, Yoshimi M, et al. A framework for ODE-based multimodal biochemical simulations on an FPGA. Proceedings of the 2005 International Conference on Field Programmable Logic and Applications (FPL); 2005:574-577.

[12] Fasih A, Trong TD, Chedjou JC, Kyamakya K. New computational modeling for solving higher order ODE based on FPGA. Proceedings of the 2nd International Workshop on Nonlinear Dynamics and Synchronization; 2009:49-53.

[13] P. Ravindra Babu, Usha Pal, R.N. Sen, R. Karthikeyan, Runge–Kutta type-2 method for solving reactor point kinetics equations and its validation by analyzing thermal and fast reactor benchmarks, Annals of Nuclear Energy, Volume 170, 2022,108979,ISSN:0306-4549,https://doi.org/10.1016/j.anucene.2022.108979.

[14] Faranak Rabiei and Fudziah Ismail, "Third-Order Improved Runge-Kutta Method for Solving Ordinary Differential Equation," International Journal of Applied Physics and Mathematics vol. 1, no. 3, pp. 191-194,2011.

Authorized licensed use limited to: The University of Toronto. Downloaded on April 20,2026 at 06:36:20 UTC from IEEE Xplore. Restrictions apply.